



Universidade do Minho

Braga, Portugal

TRABALHO PRÁTICO - RELATÓRIO

COMPUTAÇÃO GRÁFICA

Fase III - Curvas, Superfícies cúbicas e VBOs

Grupo 37

Engenharia Informática 2025/26

Equipa de Trabalho:

A106936 - Duarte Escairo Brandão Reis Silva

A106932 - Luís António Peixoto Soares

A106856 - Tiago Silva Figueiredo

A104704 - Inês Ferreira Ribeiro

1 Maio 2026

Índice

1. Introdução	1
2. Generator	2
2.1. Fita de Möbius	2
2.2. Garrafa de Klein	3
2.3. Superfície de Bezier	4
3. Engine	6
3.1. Vertex Buffer Objects (VBOs)	6
3.2. Rotação com atributo temporal	6
3.3. Curva de Catmull-Rom	7
3.4. Câmara Orbital	8
3.5. Câmara em Primeira Pessoa	8
4. Geração do Sistema Solar Dinâmico	11
5. Resultados Obtidos	14
6. Conclusão	15

1. Introdução

No âmbito da unidade curricular de Computação Gráfica, foi proposto o desenvolvimento de um projeto cujo objetivo consistiu na implementação de um mini motor gráfico 3D baseado numa estrutura de *scene graph*, o **engine**, e um gerador de modelos 3D, o **generator**, que fornece modelos de exemplo a serem utilizados pelo motor.

Nesta terceira fase do projeto, foi adicionado um novo tipo de modelo ao **generator** baseado em superfícies de Bezier e atualizámos o programa **engine** de modo a que o mesmo suportasse animações definidas por curvas Catmull-Rom que são percorridas ao longo de um dado intervalo de tempo, podendo o objeto estar alinhado, ou não, à curva. Nos nodos de rotação, foi também adicionada a opção de substituir o ângulo por tempo, que corresponde ao número de segundos que o objeto deve levar a fazer uma rotação completa em torno do eixo dado. Para além disso, foi desenvolvida uma cena de demonstração com uma representação dinâmica do sistema solar que, para além do sol, os planetas e algumas luas, inclui um cometa com uma trajetória definida por uma curva Catmull-Rom.

Como extras, o **generator** passa a conseguir gerar modelos de fitas de Möbius, de garrafas de Klein e de esferas com ruído, e o **engine** é capaz de alternar entre duas câmeras distintas, uma câmara orbital e uma câmara em primeira pessoa.

2. Generator

2.1. Fita de Möbius

A fita de Möbius é uma superfície não orientável, ou seja, uma superfície que possui apenas um lado e que, ao ser percorrida, se permite passar de um lado para outro sem cruzar nenhuma borda. Esta primitiva foi implementada como trabalho extra e pode ser gerada através do seguinte comando:

```
./generator mobiusStrip <radius> <width> <slices> <dest file>
```

A parametrização desta superfície utiliza dois parâmetros contínuos: o ângulo $u \in [0, 2\pi]$ que controla a posição ao longo do anel, e $v \in [-width, width]$, que controla a posição ao longo da largura da fita.

A superfície é constituída por $(slices + 1) \cdot 2$ pontos, onde, para cada ponto, as coordenadas cartesianas vão ser calculadas através das seguintes equações paramétricas:

$$x = \left(radius + v \cdot \cos\left(\frac{u}{2}\right) \right) \cdot \cos(u)$$

$$y = \left(radius + v \cdot \cos\left(\frac{u}{2}\right) \right) \cdot \sin(u)$$

$$z = v \cdot \sin\left(\frac{u}{2}\right).$$

, onde $radius$ é o raio do anel central da fita. O valor $\frac{u}{2}$ usado como argumento nas funções trigonométricas é o responsável pela torção encontrada na superfície: ao percorrer o anel completo, acontece uma rotação de 180° , o que faz com que a fita regresse ao ponto de partida com a orientação invertida.

Na implementação desta superfície constrói-se uma grelha de vértices de dimensão $(slices + 1) \cdot 2$, onde, para cada ponto da grelha, o valor de u é calculado através da expressão $u = i \cdot \frac{2\pi}{slices}$, sendo i o índice da linha em que o ponto a calcular se encontra na grelha. Já o valor de v varia entre $-width$ e $+width$, dependendo da coluna onde o ponto se encontra.

Tendo sido calculados todos os pontos da grelha, onde i representa o índice da linha e j o índice da coluna, é possível organizar os vértices dividindo a grelha em quadrados, sendo cada um constituído por dois triângulos. Cada quadrado é formado pelos vértices nas posições (i, j) , $(i, j + 1)$, $(i + 1, j)$ e $(i + 1, j + 1)$, sendo este processo aplicado a todos os pontos da grelha, com a exceção da última linha e da última coluna. Para assegurar o fecho correto da torção da fita, a última linha requer um tratamento especial: em vez dos quatro vértices habituais, os dois últimos pontos são substituídos pelos vértices da primeira linha com as colunas trocadas, ou seja, são usados os pontos (i, j) , $(i, j + 1)$, $(0, 1)$ e $(0, 0)$.

A natureza não orientável da figura exige um tratamento distinto na geração dos triângulos. Devido à superfície não possuir um lado interior e exterior bem definidos, uma vez que possui apenas um lado, cada triângulo deve ser gerado duas vezes com a ordem dos vértices oposta, de forma a garantir que a superfície é visível independentemente da direção em que é observada.

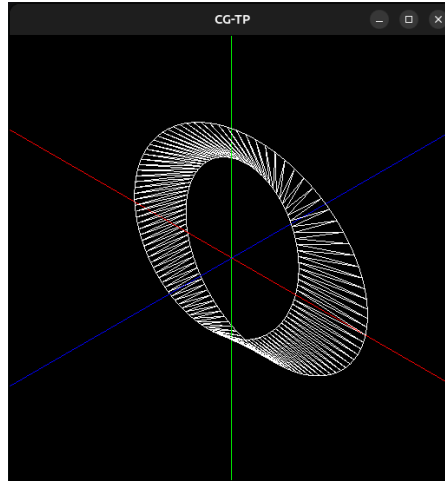


Figura 1: Resultado da geração e renderização da fita de Möbius

2.2. Garrafa de Klein

A garrafa de Klein é, tal como a fita de Möbius, uma superfície não orientável. No entanto, ela distingue-se da anterior por ser uma superfície fechada e sem fronteira, ou seja, sem borda. Tal como a fita de Möbius, esta primitiva foi implementada como um extra e pode ser gerada através do seguinte comando:

```
./generator kleinBottle <slices> <stacks> <dest file>
```

A parametrização desta figura utiliza dois ângulos $u \in [0, 2\pi]$ e $v \in [0, 2\pi]$, e pode variar entre duas formas distintas dependendo do valor do ângulo u . Esta distinção é necessária para descrever a auto-inserção apresentada pela garrafa de Klein quando representada tridimensionalmente.

Esta superfície é constituída por $(\text{slices} + 1) \cdot (\text{stacks} + 1)$ pontos, onde, para $u \in [0, \pi[$, as coordenadas de cada ponto são calculadas por:

$$x = 3 \cdot \cos(u) \cdot (1 + \sin(u)) + \left(2 \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \cos(u) \cdot \cos(v)$$

$$y = 8 \cdot \sin(u) + \left(2 \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \sin(u) \cdot \cos(v)$$

$$z = \left(2 \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \sin(v).$$

Para $u \in [\pi, 2\pi]$, o cálculo de y simplifica-se e o componente $\cos(u) \cdot \cos(v)$ usado no cálculo do valor de x é substituído por $\cos(v + \pi)$, de forma a finalizar a torção e permitir que a superfície se feche sobre si própria:

$$x = 3 \cdot \cos(u) \cdot (1 + \sin(u)) + \left(2 \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \cos(v + \pi)$$

$$y = 8 \cdot \sin(u)$$

$$z = \left(2 \cdot \left(1 - \frac{\cos(u)}{2}\right)\right) \cdot \sin(v).$$

Na implementação desta superfície constrói-se uma grelha de vértices de dimensão $(\text{slices} + 1) \cdot (\text{stacks} + 1)$, onde, para cada posição, os valores de u e v são calculados

através das expressões $u = i \cdot \frac{2\pi}{\text{slices}}$ e $v = j \cdot \frac{2\pi}{\text{stacks}}$, sendo i e j os índices da linha e coluna em que a posição que se está a calcular se encontra na grelha, respetivamente.

Seguindo o mesmo padrão da fita de Möbius, os triângulos da superfície são gerados decompondo a grelha em quadrados. Cada quadrado é formado pelos vértices nas posições (i, j) , $(i, j + 1)$, $(i + 1, j)$ e $(i + 1, j + 1)$, sendo este processo aplicado a todos os pontos da grelha, com a exceção da última linha e da última coluna. Devido à garrafa de Klein também ser uma superfície não orientável, cada triângulo também deve ser gerado duas vezes com a ordem dos vértices opostas, seguindo os mesmos princípios da fita de Möbius.

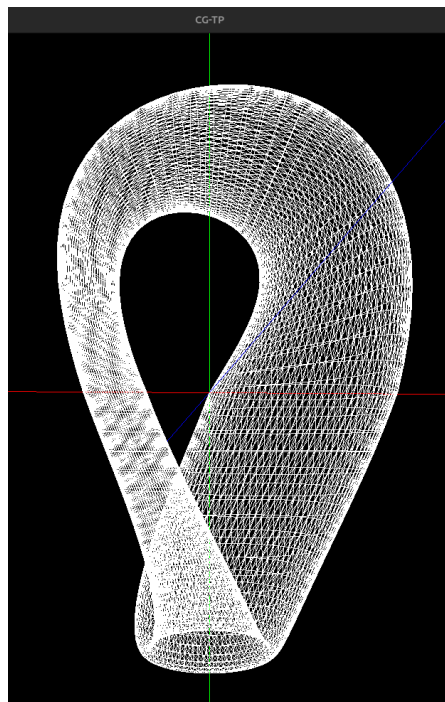


Figura 2: Resultado da geração e renderização da garrafa de Klein

2.3. Superfície de Bezier

A superfície de Bezier é uma extensão das curvas cúbicas de Bezier. Tal como acontece nas curvas, a superfície de Bezier é definida a partir de um conjunto de pontos de controlo. Ao contrário das primitivas vistas anteriormente, os parâmetros usados na construção desta superfície estão guardados num ficheiro de texto com extensão *.patch*, que contém informação referente aos patches e respetivos pontos de controlo usados para construir a primitiva. O nível de tesselação é o único parâmetro passado como argumento ao *generator*. Os patches usados neste programa são patches bicúbicos, o que significa que cada patch vai ter 16 pontos de controlo. Esta superfície pode ser gerada através do comando:

```
./generatorpatch <patch file> <tessellation> <dest file>
```

Cada ponto da superfície é calculado através da equação paramétrica de Bezier bicúbica, que combina dois parâmetros $u \in [0, 1]$ e $v \in [0, 1]$ com os 16 pontos de controlo do respetivo patch, guardados numa matriz $4 \cdot 4$:

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_{i(u)} \cdot B_{j(v)} \cdot p_{ij}$$

, onde $B_{i(u)}$ e $B_{j(v)}$ são os polinômios de Bernstein de grau 3, definidos por:

$$B_0(n) = (1 - n)^3, B_1(n) = 3 \cdot n \cdot (1 - n)^2, B_2(n) = 3 \cdot n^2 \cdot (1 - n), B_3(n) = n^3$$

, e p_{ij} é o ponto de controlo na posição (i, j) da matriz.

A implementação começa por fazer parsing do ficheiro <patch file>, onde são lidos o número de patches, os índices dos pontos de controlo de cada patch e também as coordenadas dos pontos de controlo. Para cada patch é gerada uma grelha de vértices de dimensão $(\text{tessellation} + 1) \cdot (\text{tessellation} + 1)$, sendo os valores de u e v calculados a partir do $\delta = \frac{1}{\text{tessellation}}$, de tal forma que $u_{ij} = i \cdot \delta$ e $v_{ij} = j \cdot \delta$.

Para cada posição (i, j) , o ponto correspondente na superfície é $S(u_{ij}, v_{ij})$, que é calculado com os 16 pontos de controlo do patch. Assim como visto na fita de Möbius e na garrafa de Klein, os triângulos da superfície são gerados decompondo a grelha em quadrados. Cada quadrado é formado pelos vértices nas posições (i, j) , $(i, j + 1)$, $(i + 1, j)$ e $(i + 1, j + 1)$, sendo este processo aplicado a todos os pontos da grelha, com a exceção da última linha e da última coluna.

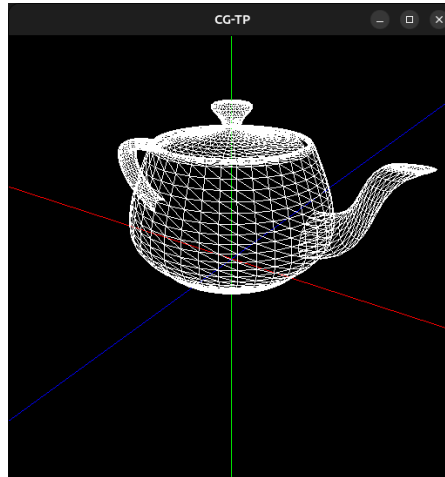


Figura 3: Resultado da geração e renderização de uma superfície de Bezier usando os pontos de controlo do Teapot

3. Engine

3.1. Vertex Buffer Objects (VBOs)

Os Vertex Buffer Objects, ou VBOs, são fundamentais para a otimização da transferência de dados geométricos entre a memória principal do computador e a unidade de processamento gráfico (GPU). Um VBO é um pedaço de memória que é alocado na GPU e que serve para armazenar vértices de primitivas geométricas, de forma a que a GPU seja capaz de aceder a estes vértices de forma direta sem a necessidade de haver repetidas transferências de dados. O uso de VBOs pode levar a ganhos significativos no desempenho, especialmente quando se trata de cenas complexas que envolvam um elevado número de triângulos ou outros polígonos.

Assim como foi visto na fase anterior, as cenas representadas pelo *engine* são armazenadas em hierarquias de grupos, onde cada grupo pode conter várias transformações geométricas e primitivas. Na implementação desta fase, cada grupo possui uma instância de um VBO, onde o mesmo é usado para armazenar de forma contígua todos os vértices contidos nos ficheiros modelo das primitivas pertencentes a esse grupo.

Na fase inicial do programa, durante o parsing do ficheiro *.xml* passado como argumento ao *engine*, para cada grupo iterado, os ficheiros de modelo são lidos e os vértices contidos neles são armazenados num buffer temporário. Para cada ficheiro de modelo, são armazenados também o número de vértices contidos nesse ficheiro. Estes contadores são essenciais na hora de renderizar a cena, uma vez que permitem saber exatamente quando começam e quando acabam os vértices de cada primitiva no buffer onde os mesmos estão armazenados.

Após todos os vértices de todas as primitivas do grupo terem sido recolhidos, a instância do VBO do grupo é efetivamente alocada na memória da GPU. Após isso, os vértices armazenados no buffer temporário são copiados para o VBO alocado na GPU, utilizando um padrão de escrita estático.

Durante a fase de renderização, assim como descrito na fase 2, a hierarquia de grupos é iterada, e, para cada grupo, o sistema itera sobre os contadores de vértices do grupo e determina o vértice inicial e o número de vértices a desenhar para cada primitiva. Conhecendo a posição inicial no buffer e o número de vértices a desenhar, o VBO do grupo é ativado e a primitiva é desenhada. Tal como na fase anterior, as transformações geométricas são aplicadas antes da renderização, de forma a afetar todas as primitivas de forma consistente.

3.2. Rotação com atributo temporal

Esta rotação com atributo temporal que é pedida na fase 3 no trabalho prático implementa uma rotação contínua de 360° em relação a um dado eixo que deve acontecer num intervalo de tempo passado como parâmetro. Esta transformação recebe como parâmetros um vetor tridimensional (x, y, z) que define o eixo no qual a rotação deve acontecer e o intervalo de tempo em segundos que define o tempo de uma volta completa.

Ao renderizar a cena, o *engine* consulta o tempo global decorrido desde o arranque do programa, obtido através do GLUT em milissegundos e depois convertido para segundos. Após obter esse tempo, é calculada a velocidade angular em unidades de

graus por segundo através da expressão $\frac{360}{\Delta t}$, sendo Δt o intervalo de tempo passado como parâmetro. Multiplicando a velocidade angular pelo tempo decorrido permite-nos obter o ângulo acumulado e ao aplicar uma operação de redução módulo 360 a esse valor, somos capazes de obter um ângulo equivalente contido no intervalo $[0, 360]$. Por fim, o *engine* usa o ângulo resultante e o eixo passado como parâmetro para aplicar uma rotação ao eixo de coordenadas do OpenGL e criar o efeito pretendido.

O efeito visual de rotação contínua é criado devido ao facto da função de renderização ser continuamente invocada. Em cada chamada, o tempo global decorrido é obtido, o ângulo resultante é recalculado e a rotação é reaplicada ao eixo do OpenGL. Este processo cria a ilusão de um movimento contínuo à volta do eixo, uma vez que, a cada invocação, o ângulo usado na rotação aumenta ligeiramente.

3.3. Curva de Catmull-Rom

Uma curva de Catmull-Rom é definida por quatro ou mais pontos de controlo pelos quais passa e que utiliza para fazer a interpolação da posição dos pontos intermédios e do vetor tangente.

A posição de um dado ponto, no segmento da curva entre os pontos de controlo p_n e p_{n+1} , é obtida do seguinte produto:

$$p(t) = (t^3 \ t^2 \ t \ 1) \begin{pmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1 & -2.5 & 2 & -0.5 \\ -0.5 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} p_{n-1} \\ p_n \\ p_{n+1} \\ p_{n+2} \end{pmatrix} \quad (1)$$

Nesta equação, t corresponde à localização específica do ponto no segmento, esta é calculada em função do tempo decorrido desde que o programa iniciou, obtido por `glutGet(GLUT_ELAPSED_TIME)`, e do tempo definido, no ficheiro de configuração, para o objeto completar a curva.

Para aqueles objetos que devem percorrer a trajetória alinhados à curva, é calculado também o vetor tangente, que é dado pela derivação da sua posição, $\frac{p(t)}{dt}$, ou seja:

$$p'(t) = (3t^2 \ 2t \ 1 \ 0) \begin{pmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1 & -2.5 & 2 & -0.5 \\ -0.5 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} p_{n-1} \\ p_n \\ p_{n+1} \\ p_{n+2} \end{pmatrix} \quad (2)$$

O vetor resultante é normalizado e usado, através do produto vetorial, para calcular os vetores que vão compor a matriz de rotação, M , usada para alinhar o objeto.

$$\vec{X} = \frac{p'(t)}{|p'(t)|} \quad (3)$$

$$\vec{Z} = \vec{X} \times \overrightarrow{up} \quad (4)$$

$$\vec{Y} = \vec{Z} \times \vec{X} \quad (5)$$

$$M = \begin{pmatrix} X_x & Y_x & Z_x & 0 \\ X_y & Y_y & Z_y & 0 \\ X_z & Y_z & Z_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (6)$$

Depois de executar o programa **engine**, se houver uma Curva de Catmull-Rom a ser renderizada, a trajetória da curva também é renderizada. Ao clicar na tecla **H** a trajetória da curva deixa de ser renderizada, e ao clicar na tecla **R** a trajetória volta a ser renderizada.

3.4. Câmara Orbital

A câmara orbital permite observar uma cena a partir de um ponto que orbita em torno de um objeto ou ponto fixo. A mesma pode ser ativada ao clicar na tecla **1** e desativada com a tecla **0** que restaura a câmara base definida no ficheiro com extensão *.xml* passado como argumento ao **engine**. Nesta implementação a câmara orbital está sempre focada na origem do referencial.

A câmara orbital é representada por 3 parâmetros:

- Raio (r) : Distância da câmara à origem
- Ângulo azimutal(α) : Ângulo de rotação horizontal em torno do eixo y
- Ângulo de elevação(β) : Ângulo de rotação vertical

Para obter a posição da câmara no referencial, esses parâmetros são convertidos para coordenadas cartesianas através das equações:

$$x = r \cdot \cos(\beta) \cdot \sin(\alpha)$$

$$y = r \cdot \sin(\beta)$$

$$z = r \cdot \cos(\beta) \cdot \cos(\alpha).$$

A rotação horizontal é controlada pelas teclas **direcionais esquerda** e **direita**, que incrementam e decrementam α respetivamente, enquanto que a rotação vertical é controlada pelas teclas **cima** e **baixo**, que controlam o β , sendo este limitado ao intervalo $[-1.5, 1.5]$ para evitar a inversão da câmara. O zoom é controlado pelas teclas **Page Up** e **Page Down** que incrementam e decrementam o valor do raio r , sendo o valor mínimo do mesmo 0.1 para evitar que a câmara se aproxime demasiado da origem. A velocidade do zoom pode ainda ser ajustada através das teclas **+** e **-**.

Quando um dos parâmetros da câmara é alterado, as coordenadas cartesianas são recalculadas e a posição da câmara no referencial é atualizada, mantendo o foco da câmara na origem do referencial.

3.5. Câmara em Primeira Pessoa

A câmara em primeira pessoa permite ao utilizador andar livremente pela cena. A mesma pode ser ativada ao clicar na tecla **2** e desativada com a tecla **0** que restaura a câmara base, assim como foi visto na secção anterior.

A movimentação da câmara pela cena deve-se a dois vetores:

- $\vec{d}$: Direção do olhar da câmara
- $\vec{r}$: Direção perpendicular ao vetor $\vec{d}$ no plano horizontal

Ao contrário da câmara orbital onde o foco era fixo na origem do referencial, numa câmara em primeira pessoa tanto a posição quanto o foco da câmara mudam à medida que o utilizador se move pela cena.

A posição da câmara pode ser calculada através das seguintes equações:

- Movimento para frente: $P_{\text{new}} = P_{\text{old}} + v \cdot \vec{d}$
- Movimento para trás: $P_{\text{new}} = P_{\text{old}} - v \cdot \vec{d}$
- Movimento lateral esquerdo: $P_{\text{new}} = P_{\text{old}} + v \cdot \vec{r}$
- Movimento lateral direito: $P_{\text{new}} = P_{\text{old}} - v \cdot \vec{r}$

, onde v é a velocidade do movimento.

O vetor de direção $\vec{d}$ é calculado através da orientação da câmara, cuja mesma é definida pelos ângulos $yaw(\alpha)$ e $pitch(\beta)$, que são responsáveis pela rotação horizontal e vertical, respetivamente. Para prevenir a inversão da câmara, neste caso quando o ângulo $pitch(\beta)$ atinge valores de 90° ou -90° , o valor de β foi limitado ao intervalo $[-89^\circ, 89^\circ]$.

O vetor de direção $\vec{d}$ pode ser calculado através das seguintes equações:

$$\begin{aligned}\vec{d}_x &= \sin(\alpha) \cdot \cos(\beta) \\ \vec{d}_y &= \sin(\beta) \\ \vec{d}_z &= \cos(\alpha) \cdot \cos(\beta)\end{aligned}$$

Já o vetor $\vec{r}$, perpendicular a $\vec{d}$, pode ser calculado através das seguintes equações:

$$\begin{aligned}\vec{r}_x &= \cos(\alpha) \cdot \cos(\beta) \\ \vec{r}_y &= 0 \\ \vec{r}_z &= -\sin(\alpha) \cdot \cos(\beta)\end{aligned}$$

As coordenadas do ponto de foco da câmara também dependem do vetor $\vec{d}$, e podem ser calculadas a partir da seguinte equação:

$$L = P + \vec{d}$$

A movimentação da câmara é controlada pelas teclas **W**, **S**, **A** e **D**, que deslocam a câmara para frente, para trás, para a esquerda e para a direita, respetivamente. As teclas **Space** e **Tab** podem ser usadas para incrementar e decrementar a componente do eixo y da posição. A velocidade do movimento pode ser ajustada com as teclas **+** e **-**.

A rotação da câmara é controlada através do **rato**. Ao clicar em qualquer botão do rato e arrastar o mesmo pela cena, o deslocamento horizontal e vertical alteram os valores

dos ângulos *yaw* e *pitch* respetivamente. A atualização dos parâmetros da câmara, tais como a posição P e as coordenadas do ponto de foco L , é feita aproximadamente 60 vezes por segundo, o que garante que o utilizador seja capaz de se movimentar de forma suave e fluída pela cena.

4. Geração do Sistema Solar Dinâmico

Para esta fase do trabalho, o modelo estático do sistema solar foi transformado num modelo dinâmico, permitindo a simulação do movimento dos corpos celestes e do fenómeno de rotação de alguns destes corpos. As novas transformações definidas nesta fase foram fundamentais para a evolução do modelo, a Curva de Catmull-Rom permitiu simular os movimentos de translação dos planetas e do cometa, e a rotação com atributo temporal permitiu simular a rotação natural apresentada por alguns dos corpos celestes.

A combinação destas duas transformações cria um sistema solar dinâmico mais perto da realidade. Cada planeta orbita em torno do sol, ao mesmo tempo que gira ao redor de si mesmo, simulando os fenómenos físicos de translação e rotação. Para os corpos celestes com satélites naturais, como a Terra, a hierarquia de transformações garante que os satélites seguem tanto o movimento orbital do planeta pai, além também do seu próprio movimento de translação e rotação, permitindo a criação de um sistema complexo e coerente.

Abaixo é possível ver um exemplo da utilização das novas transformações na representação de um modelo em xml:

```
<!-- SOL -->
<group>
  <transform>
    <scale x="5.5" y="5.5" z="5.5" />
    <rotate time="90" x="0" y="1" z="0" />
  </transform>
  <models>
    <model file="sphere_1_10_10.3d" />
  </models>

  <!-- TERRA -->
  <group>
    <transform>
      <translate time="16" align="true">
        <point x="-2.37" y="0" z="1.99" />
        <point x="-3.08" y="0" z="-0.27" />
        <point x="-1.99" y="0" z="-2.37" />
        <point x="0.27" y="0" z="-3.08" />
        <point x="2.37" y="0" z="-1.99" />
        <point x="3.08" y="0" z="0.27" />
        <point x="1.99" y="0" z="2.37" />
        <point x="-0.27" y="0" z="3.08" />
      </translate>
      <scale x="0.2" y="0.2" z="0.2" />
      <rotate time="4" x="0" y="1" z="0" />
    </transform>
    <models>
      <model file="sphere_1_10_10.3d" />
    </models>

    <!-- Lua -->
    <group>
      <transform>
        <translate time="3" align="true">
```

```

    <point x="2.300" y="0" z="0.000" />
    <point x="1.626" y="0" z="1.626" />
    <point x="0.000" y="0" z="2.300" />
    <point x="-1.626" y="0" z="1.626" />
    <point x="-2.300" y="0" z="0.000" />
    <point x="-1.626" y="0" z="-1.626" />
    <point x="0.000" y="0" z="-2.300" />
    <point x="1.626" y="0" z="-1.626" />
  </translate>
  <scale x="0.2" y="0.2" z="0.2" />
  <rotate time="7" x="0" y="1" z="0" />
</transform>
<models>
  <model file="sphere_1_10_10.3d" />
</models>
</group>
</group>
</group>

```

No exemplo mostrado, é possível ver que o sol possui uma rotação com atributo temporal para simular a rotação sobre si mesmo. No caso da Terra e da Lua, ambos possuem tanto uma Curva de Catmull-Rom como a nova rotação, uma vez que ambos possuem movimentos de translação e rotação. Devido à hierarquia de transformações vista na última fase, a Lua vai herdar o movimento orbital da Terra, além de possuir o seu próprio através das novas transformações.

Na figura a seguir é possível ver a demonstração construída nesta fase:

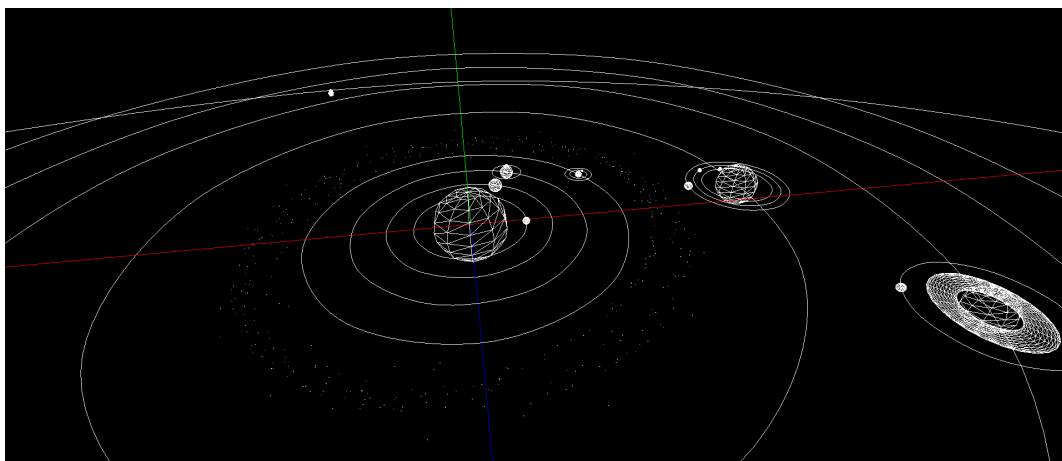


Figura 4: Parte do modelo dinâmico do Sistema Solar com as curvas de catmull-rom visíveis

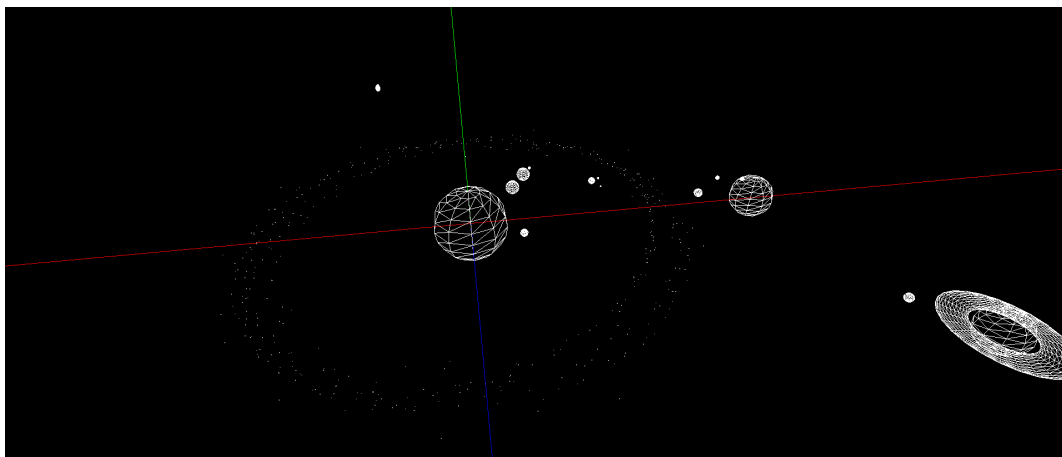


Figura 5: Parte do modelo dinâmico do Sistema Solar com as curvas de catmull-rom não visíveis

Na próxima fase, iremos atualizar o **generator** e o **engine** de forma a a criar um Sistema Solar dinâmico com iluminação e texturas.

5. Resultados Obtidos

Assim como nas fases 1 e 2, a equipa docente disponibilizou outputs de referência de modo a conseguirmos comparar os nossos resultados. Nas seguintes figuras é possível ver a comparação entre os outputs obtidos e os disponibilizados:

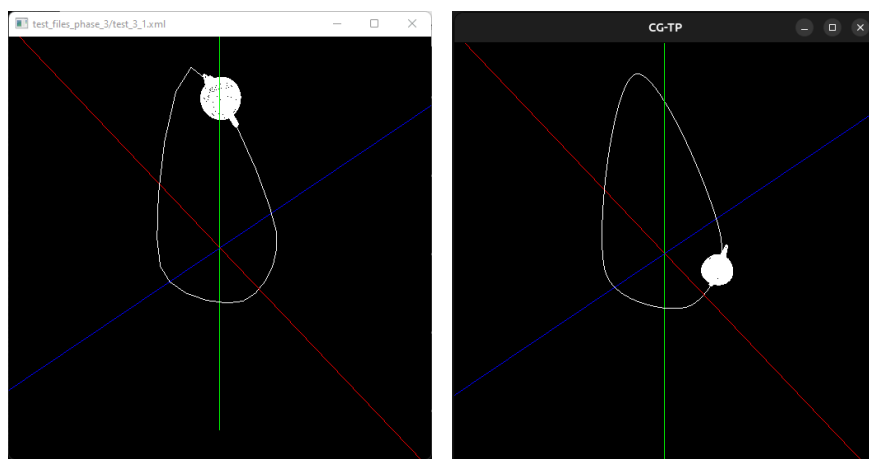


Figura 6: Comparação dos outputs obtidos no teste 1

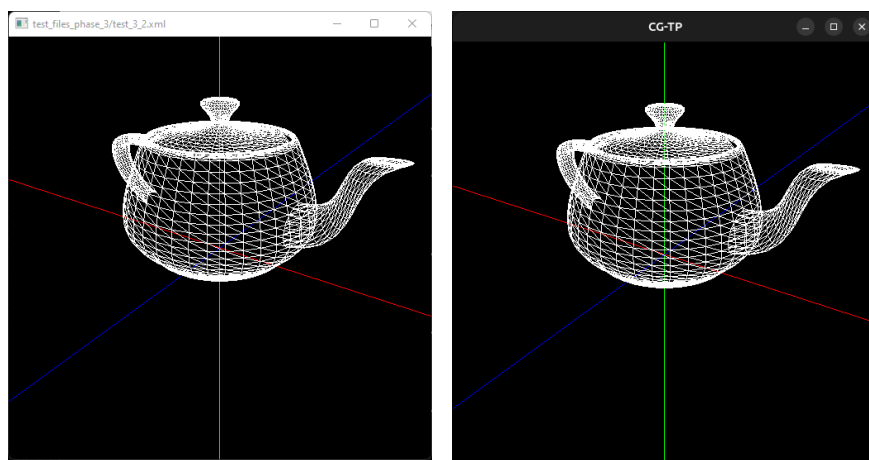


Figura 7: Comparação dos outputs obtidos no teste 2

6. Conclusão

A realização da terceira fase deste projeto permitiu aprofundar a nossa compreensão de duas das principais curvas cúbicas que aprendemos na Unidade Curricular de Computação Gráfica. Sendo as curvas de Bézier usadas para criar Superfícies de Bézier, splines matemáticas que permitem modelar figuras mais detalhadas mantendo um aspeto suave e contínuo, e as curvas de Catmull-Rom utilizadas para construir trajetórias complexas que passam pelos pontos de controlo dados.

Graças a estas, conseguimos melhorar o modelo de demonstração da fase anterior, tornando-o dinâmico com os planetas do Sistema Solar a girar, não só sobre o seu próprio eixo mas também a percorrer a rota em torno do Sol com diferentes velocidades, e adicionando um cometa com um modelo mais complexo e um trajeto intrincado.

Nesta fase, os modelos passaram a ser desenhados usando VBOs sem índices que, ao contrário do modo imediato, não chamam a função `glVertex` para cada vértice e conseguem tirar proveito do potencial paralelismo disponibilizado pelo GPU, melhorando imenso o desempenho do programa.

Este desempenho é extremamente necessário nesta etapa, onde para além da adição do movimento às cenas 3D, foram também adicionados, como extras, modelos elaborados de fitas de Möbius e de garrafas de Klein, e a possibilidade de alternar entre dois tipos de câmaras.